DAGSTER · DATABRICKS SQL · JSON · KAFKA



Jobotron: the RFC’s upstream producer, rebuilt on the team’s real contract

Candidate selection on the team’s Databricks lakehouse — the notebooks’ SQL verbatim, one JSON message per customer keyed by CustomerId, one market per topic.

THREE INSTANCES: [CY, GR, RO] // VERIFIED LIVE 26.10.2026 // CLEARANCE: Open

The notebooks’ queries, published per market



TL;DR EXECUTION SUMMARY

The notebooks’ SQL, token-verbatim. Submitted to the team’s SQL warehouse in its catalog.schema context — seconds-class statements, real candidates.

One JSON message per customer. The 13-field PascalCase envelope; `ItemPayloadHash` = sha256 over pinned canonical JSON; `ItemId` = `<hash>-<number>`, the dedup token.

Dry-run by default. Every run scans, builds and hashes and sends nothing — publishing is a deliberate per-run override. Dedup and re-validation stay with the worker.

The use case, per the contract

> Scheduling and candidate selection remain externalized to upstream producers. Such producers can be scheduled Databricks jobs that query Data Lake to identify eligible customers and publish individual messages to the relevant operator topic.
>
> — Jobotron RFC, 08.10.2026 (Accounts & Payments / KYC solutions)

The Jobotron worker

Per-operator Kafka consumers, a Job Status WebAPI and a state database — Redis idempotency, eligibility re-validation, Iris / CustomerService actions. **Owned by Accounts & Payments.**

This component — the upstream producer

Scheduled candidate selection + publishing on Dagster, one instance per market job. **Owned by Big Data.** The review thread asked for exactly this: *“Could the RFC name Dagster as the scheduler and publisher?”*

✉ **The unit of work:** one Kafka message per selected customer. The worker checks `ItemId` for not-processed-before, re-validates eligibility against live data, executes the actions and records the outcome.

Three market instances, one minimal YAML each

orchestration/defs/unverified_customers_gr/defs.yaml

```yaml
# the GR notebook as a Dagster component instance:
# one query type x one operator (D9), six typed facts.
type: orchestration.components.
      reverse_etl_jobotron.JobotronProducer

attributes:
  query_type: unverified_customers
  operator: gr
  job_name: Send_Email_to_NonVerified_GR_Customers
  catalog: de
  schema_name: voperdb
  query: |
    # the notebook's SQL, token-verbatim --
    # qualified to the market's physical shard
```

1 Everything derives

Topic `jobotron_unverified_customers_gr`, assets `_setup / publish / _consume`, group, reset job — all from the six facts. No routing layer, no templates.

2 dry_run defaults to true

The component default (not a YAML fact) — the first run of a new instance can never send anything.

3 cy · gr · ro

Identical shape, one folder per market: `unverified_customers_{cy,gr,ro}` — market isolation by construction, not by routing.

The real query, qualified to the physical shard

the GR notebook's candidate query (token-verbatim)

```
select cusCustomerID as CustomerId, cusCompanyID as CompanyId
from de.voperdb_gr.cCustomer
inner join de.metadatadb.mVerificationStatus
       on cusVerificationStatusID = vstVerificationStatusID
left join de.voperdb_gr.cCustomer_Group
       on ccgCustomerID = cusCustomerID and ccgGroupID = 1425
where cusCreated between current_timestamp() - interval 28 days
                     and current_timestamp() - interval 26 days
  and cusTypeID not in (2, 5, 6, 10, 11, 15)
  and cusCompanyID = 1    -- GML(GREECE) -- guard
  and cusStatusID = 3     -- REAL -- guard
  and vstVerifiedFL = false
  and (ccgValue0 is null or ccgValue0 <> 1)
```

Why the shard

`de.voperdb.cCustomer` is a ~29-way UNION view across market schemas. Each physical shard holds exactly one company: `cy→7, gr→1, ro→4` (verified live 2026-10-26).

Same candidates, seconds-class

Qualifying to `de.voperdb_gr` keeps the notebook's semantics (the company/group filters stay as guards) and turns the UNION scan into a **5.1 s statement** instead of a queue-bound scan.

Dated deviation, recorded: docs/poc/JOBOTRON_CONTRACT_PARITY.md carries the evidence and the reasoning.

The 13-field JSON envelope, hashed so the worker can verify

a real message on the wire (run e3e71a73, item 1/313)


```
{
  "JobId": "Send_Email_to_NonVerified_GR_Customers_1_20261026153607",
  "JobName": "Send_Email_to_NonVerified_GR_Customers",
  "JobCreated": "2026-10-26T15:36:07Z",
  "RunId": "e3e71a73-…",
  "CompanyId": 1,
  "CustomerId": "488101304",
  "ItemsCount": 313,
  "ItemNumber": 1,
  "ItemId": "63cb9003d855…7420f-1",
  "ItemPayload": { "CustomerId": 488101304, "CompanyId": 1 },
  "ItemPayloadHash": "63cb9003d855…7420f",
  "IsDryRun": false,
  "SchemaVersion": 1
}
```


Canonical JSON (D3)

Compact separators, keys in the result's column order, nulls explicit, non-ASCII kept — the same rendering that lands in `ItemPayload`, so the worker recomputes and checks.

ItemId = <hash>-<number>

The dedup token. The key is `CustomerId`, cast to string, exactly the contract's. New content means a new hash and so a new `ItemId`.

JSON, not Avro

The demo replaced the Avro/Apicurio path with the pinned PascalCase contract — byte-verifiable on the wire with nothing but sha256.

What one publish run does — six stages

1 Resolve the context

The YAML's `catalog.schema` goes into the Statement Execution body — without it, unqualified names resolve against the warehouse default and `cCustomer` 404s.

2 Submit verbatim

The YAML query, token for token, on the team's warehouse. 50 s inline wait, then polling.

3 Poll, bounded

5 s cadence to a 600 s deadline; on timeout the statement is **cancelled** (this workspace's endpoint is `/cancel`) — no orphan billing, no silent hangs.

4 Build the envelope

Sort CustomerId-first, canonical JSON, sha256 per row, `ItemId`, ItemNumber order.

5 Gate on dry-run

Under the default: "built N message(s) … and sent NOTHING" — not even a Kafka connection on setup.

6 Produce & record

Key = CustomerId (string), bounded backpressure; metadata records `items_count`, `statement_seconds`, `publish_seconds`.

⚙ **Warehouse-based, seconds-class:** the default engine is `databricks_sql` — the 0-DBU engines (`delta_rs`, `delta_extension`) remain shipped behind the `scan_engine` knob, but UC-managed storage currently denies the service principal direct reads (recorded with the exact error in the parity doc).

dry_run: the blast-radius gate

Default true — in the component

Not a YAML fact: the safe default lives in `JobotronProducer` itself, so every future instance starts harmless. It gates **setup and publish**: zero side effects, not even a Kafka connection.

Per-run override

The launch config carries an optional `dry_run` — the contract's own IsDryRun is a per-run widget. An override is logged and recorded in the materialization metadata; `IsDryRun` travels in every message.

Visible in the launchpad

The per-instance config switch is re-defaulted to the YAML value, so the launchpad's scaffold shows the effective mode instead of a blank — flip and launch, or leave blank for YAML-decides.

Pin it when a market goes live

`dry_run: false` under the instance's attributes makes plain Materialize publish; delete the line to fall back to safe. cy and ro stay dry today.

✓ **Verified live:** a dry run built 13 messages and sent nothing; the live GR runs' wire messages carried `"IsDryRun": false` end to end.

The demo — built and verified live, 26.10.2026

5.1 s

candidate statement on the team's warehouse (context-verbatim, shard form)

331 items

latest run · every one keyed by CustomerId, hashed, ItemId-dedupable

1,302 msgs

on the topic across 4 live runs (313 + 328 + 330 + 331)



publish
4 live runs

jobotron_unverified_customers_gr `dedup on ItemId`
1,302 messages

Postgres mirror
331 rows — exact match

⇆ **The sliding window is the demo:** `cusCreated between now-28d and now-26d` — runs through the day went 313→328→330→331 as customers aged in. Overlapping customers keep **byte-identical ItemIds**, so dedup collapses 1,302 messages to exactly 331 mirror rows — verified in both directions.

Runs anywhere — with or without the containers

In the stack

`./bin/1_up.sh` → Dagster on :3000, the helper panel on :4002 driving setup / publish / consume / reset per market, with the per-run dry-run checkbox.

Bare, on the host

`devbox shell; dg dev` — the identical asset catalog from the project dir. `PYTHONPATH` pinned in devbox.json, `.env` auto-loaded, `POSTGRES_HOST` split per context (localhost here, host.docker.internal in compose).

▣ **Self-healing manifest:** a missing or corrupt `dbt/target/manifest.json` now regenerates via `dbt parse` at load — a wiped target used to bootstrap a 0-node skeleton that failed the whole code location; today it self-heals or fails loudly naming the cause.

✓ **Verified both ways:** the full live chain — Databricks → Kafka → mirror — ran on the host instance the same day the container stack ran it.

✉ **Same contract either way:** one component, one set of YAML folders — the runtime choice never touches the wire.

Critical trade-offs & implementation notes

Full set, no tombstones

Every run republishes whoever matches **right now**. A customer leaving the segment is silence, not a delete — the worker's dedup and re-validation make that correct by construction.

Sliding window, drifting counts

313→331 in one day is the design, not drift: each run snapshots the current window. ItemId stability across four runs is what makes the worker's dedup sound — and it held byte-for-byte.

Zero-DBU engines, blocked for now

The SP authenticates to storage but is denied reads on UC-managed `__unitystorage` paths — needs storage RBAC. The engine seam ships and waits.

OPEN QUESTIONS

storage RBAC for the SP (unlocks the 0-DBU engines) · worker-side dedup semantics at scale · schedule + alerting for the three instances · gr/ro set-level equivalence proof in a quiet warehouse window

≈5s

VERIFICATION: SECONDS-CLASS · CONTRACT-CORRECT MESSAGES · DRY-RUN-SAFE · E2E ON A LAPTOP · RUNS WITH OR WITHOUT CONTAINERS